

First normal form (1NF)

As per the rule of the first normal form, an attribute (column) of a table cannot hold multiple values. It should hold only atomic (Single) values.

Example: Suppose a company wants to store the names and contact details of its employees. It creates a table that looks like this:

emp_id	emp_name	emp_address	emp_mobile
101	Herschel	New Delhi	8912312390
102	Jon	Kanpur	8812121212 9900012222
103	Ron	Chennai	7778881212
104	Lester	Bangalore	9990000123 8123450987

Two employees (Jon & Lester) are having two mobile numbers so the company stored them in the same field as you can see in the table above.

This table is **not in 1NF** as the rule says “each attribute of a table must have atomic (single) values”, the **emp_mobile** values for employees Jon & Lester violates that rule.

To make the table complies with 1NF we should have the data like this:

emp_id	emp_name	emp_address	emp_mobile
101	Herschel	New Delhi	8912312390
102	Jon	Kanpur	8812121212
102	Jon	Kanpur	9900012222

103	Ron	Chennai	7778881212
104	Lester	Bangalore	9990000123
104	Lester	Bangalore	8123450987

Second normal form (2NF)

A table is said to be in 2NF if both the following conditions hold:

- Table is in 1NF (First normal form)
- No non-prime attribute is dependent on a proper subset of any candidate key of the table.

An attribute that is not part of any candidate key is known as **non-prime attribute**.

Example: Suppose a school wants to store the data of teachers and the subjects they teach. They create a table that looks like this: Since a teacher can teach more than one subjects, the table can have multiple rows for a same teacher.

teacher_id	subject	teacher_age
111	Maths	38
111	Physics	38
222	Biology	38
333	Physics	40
333	Chemistry	40

Candidate Keys: {teacher_id, subject}

Non prime attribute: teacher_age

The table is in 1 NF because each attribute has atomic values.

However, it is not in 2NF because non prime attribute teacher_age is dependent on teacher_id alone which is a proper subset of candidate key.

This violates the rule for 2NF as the rule says “**no** non-prime attribute is dependent on the proper subset of any candidate key of the table”.

To make the table complies with 2NF we can break it in two tables like this:

teacher_details table:

teacher_id	teacher_age
111	38
222	38
333	40

teacher_subject table:

teacher_id	subject
111	Maths
111	Physics
222	Biology
333	Physics
333	Chemistry

Now the tables comply with Second normal form (2NF).

Third Normal form (3NF)

A table design is said to be in 3NF if both the following conditions hold:

- Table **must be in 2NF**
- **Transitive functional dependency** of **non-prime attribute** on any **super key** should be **removed**.

An attribute that is not part of any **candidate key** is known as **non-prime attribute**.

In other words 3NF can be explained like this:

A table is in 3NF **if** it is in 2NF and for each functional dependency $X \rightarrow Y$ at least **one of the following** conditions hold:

- X is a **super key** of table
- Y is a **prime attribute** of table

Super key is a **single key or a group of multiple keys that can uniquely identify tuples in a table**. Super Key can contain multiple attributes that might not be able to independently identify tuples in a table, but when grouped with certain keys, they can identify tuples uniquely.

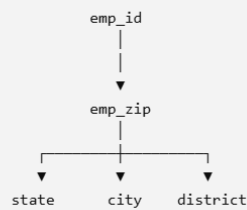
An attribute that is a **part of** one of the candidate keys is known as **prime attribute**.

Example: Suppose a **company wants to store the complete address of each employee**, they create a table named **employee_details** that looks like this:

Problem:

emp_id	emp_name	emp_zip	emp_state	emp_city	emp_district
1001	John	282005	UP	Agra	Dayal Bagh
1002	Ajeet	222008	TN	Chennai	M-City
1006	Lora	282007	TN	Chennai	Urrapakkam
1101	Lilly	292008	UK	Pauri	Bhagwan
1201	Steve	222999	MP	Gwalior	Ratan

The original table has:



The problem is:

`state`, `city`, and `district` depend on `emp_zip`, not directly on `emp_id`.

And `emp_zip` itself depends on `emp_id`.

Therefore:

`emp_id` → `emp_zip` → `emp_city`

This is a **transitive dependency**.

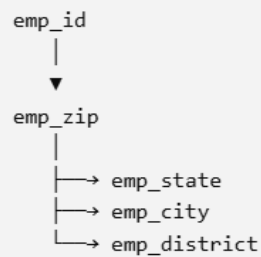
Assume:

`emp_id` → `emp_name`
`emp_id` → `emp_zip`

and

`emp_zip` → `emp_state`
`emp_zip` → `emp_city`
`emp_zip` → `emp_district`

Therefore:



This is a **transitive dependency**.

Candidate Keys: {`emp_id`}

Super keys: {`emp_id`}, {`emp_id`, `emp_name`}, {`emp_id`, `emp_name`, `emp_zip`}...so on

[**Note:** A **super key** is a set of one or more attributes (columns), which can uniquely identify a row in a table.].

Non-prime attributes: all attributes except `emp_id` are non-prime as they are not part of any candidate keys.

Attributes	
Prime	Non-Prime
emp_id	emp_name emp_zip emp_state emp_city emp_district

Here, emp_state, emp_city & emp_district dependent on emp_zip.

And, emp_zip is dependent on emp_id that makes non-prime attributes (emp_state, emp_city & emp_district) transitively dependent on super key (emp_id). This violates the rule of 3NF.

emp_id → emp_zip → emp_state, emp_city, emp_district

So, emp_state, emp_city, emp_district is indirectly dependent on emp_id.

This is the problem that 3NF tries to remove.

To make this table complies with 3NF we have to **break** the table into two tables to remove the transitive dependency:

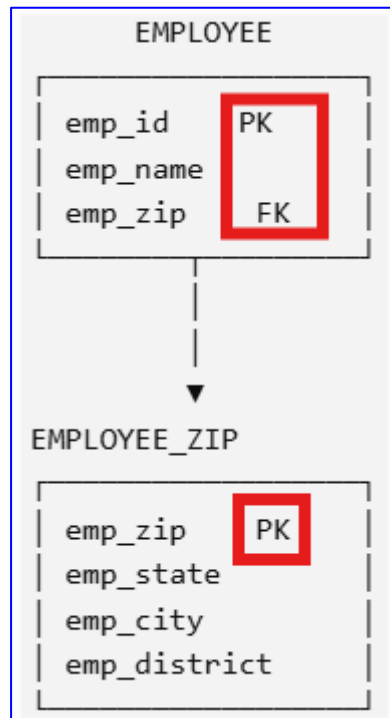
employee table:

<u>emp_id</u>	<u>emp_name</u>	<u>emp_zip</u>
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999

<u>emp_zip</u>	<u>emp_state</u>	<u>emp_city</u>	<u>emp_district</u>
282005	UP	Agra	<u>Dayal Bagh</u>
222008	TN	Chennai	M-City
282007	TN	Chennai	<u>Urrapakkam</u>
292008	UK	<u>Pauri</u>	<u>Bhagwan</u>
222999	MP	Gwalior	Ratan

Here: <pre>emp_id → emp_name emp_id → emp_zip</pre> There is no transitive dependency.	Here: <pre>emp_zip → emp_state emp_zip → emp_city emp_zip → emp_district</pre> <u>emp_zip</u> is the key of this table.
--	---

Final Structure:



What is BCNF?

A relation is in **BCNF** if:

For every functional dependency $X \rightarrow Y$, **X** must be a **super key**.

In very simple words:

3NF:

$X \rightarrow Y$

X should be a Super Key

OR

Y should be a Prime Attribute

BCNF:

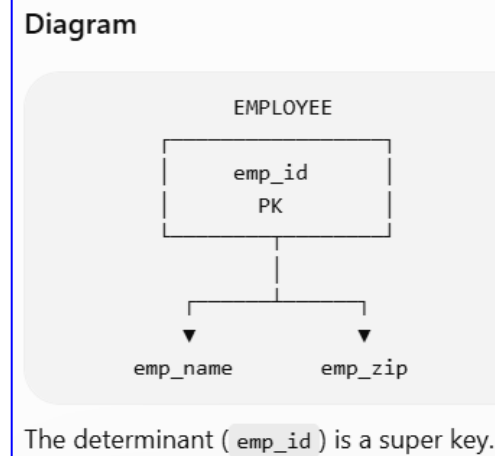
$X \rightarrow Y$

X **MUST** be a Super Key

Therefore, **BCNF** is **stricter** than **3NF**.

Example:

<u>emp_id</u>	<u>emp_name</u>	<u>emp_zip</u>
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999



Every BCNF relation is in 3NF, but **every 3NF relation is not necessarily in BCNF.**

Normal Form	Main Rule
1NF	No repeating groups; atomic values
2NF	No partial dependency
3NF	No transitive dependency
BCNF	Every determinant must be a super key

1NF : Atomic values

2NF : Whole key

3NF : No non-key $\rightarrow$ non-key

BCNF : Every determinant = Super Key